



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

Department of Computer Science
Faculty of Computing

SECP3843 – SPECIAL TOPIC IN DATA ENGINEERING

(Tutorial 2)

Creating a Simple ETL Pipeline with Apache Spark

Programme	: Bachelor of Computer Science (<i>Data Engineering</i>)
Subject Code	: SECP3843
Subject Name	: Special Topic in Data Engineering
Session-Sem	: 2025/2026-02

Prepared by : 1) Muhammad Naim bin Abdullah (A23CS0134)
2) Muhammad Adam bin Razali (A23CS0116)
3) Afif Shaqir Irfan bin Arqam (A23CS0204)

Section : 02

Lecturer : Dr Aryati binti Bakri

1.0 Business Understanding	3
1.1 Domain Context	3
1.2 Business Objectives	3
1.3 Success Criteria	3
2.0 Data Understanding	3
2.1 Data Source Description	3
2.2 Data Quality Observation	3
2.3 Relational Data Model	3
3.0 Data Preparation	3
3.1 Ingestion	3
3.2 Transformation	3
4.0 Modelling	3
4.1 System Architecture	3
4.2 Multidimensional Data Model	3
5.0 Evaluation	3
5.1 Limitation	3
5.2 Improvement	3

1.0 Business Understanding

1.1 Domain Context

The Brazilian Basic Education Census, known as Censo Escolar, is a massive yearly survey run by the government to track the country's school system. It gathers information from every public and private school in Brazil, covering everything from student details and teacher count to school resources like internet, libraries, and water. This data is released for free to the public, providing an incredibly detailed look at education across the country.

1.2 Business Objectives

1. Speed Up Research :

Instead of spending hours manually opening files, researchers should be able to get answers in seconds using a fast database

2. Ready to use dashboards :

Make the data easy to view in visualisation tools like Power BI without needing extra technical setup

1.3 Success Criteria

The pipeline will be judged successful if all of the following are true :

- All 12 dimension tables and the 1 main fact table exist in the target database and properly linked together.
- All 12 years data from 2010 until 2021 are loaded, with per-year row counts matching the source CSVs
- Make sure every data that are present is accurate for example in year 2021 will have the highest student enrollment

2.0 Data Understanding

2.1 Data Source Description

2.2 Data Quality Observation

2.3 Relational Data Model

3.0 Data Preparation

All of the preparation steps are implemented through the `run_etl.py` where it combined both the ingestion and transformation levels of processing. It is also invoked inside a Docker Container alongside other dependencies such as Spark runtime, Postgres JDBC, and Python to allow the output to be shared among them.

3.1 Ingestion

```
def main() -> None:
    DATA_DIR.mkdir(parents=True, exist_ok=True)
    for year in YEARS:
        download_year(year)
    print("[done] downloads complete", flush=True)
```

Before Spark reads the data, the raw data from INEP releases must be downloaded, unzipped, and normalized into a predictable filename based on the year of the records. By default, the data is pulled from the year 2010 until 2021 and stored inside `data/*.csv` path. This process simplifies the querying of the data for ingestion.

3.2 Transformation

CSV to Parquet Conversion

```
data_csv = (  
    spark.read.format("csv")  
    .option("header", "true")  
    .option("inferSchema", "true")  
    .option("delimiter", ";")  
    .option("encoding", "latin1")  
    .load("/app/data/*.csv")  
)
```

Then the stored data let's Spark pickup through it by the number of years, then encode it with latin 1 encoding and semicolon(;) delimiter to read the data row by row based on the column available.

```
parquet_path = "/app/data/censo_escolar.parquet"  
log(f"writing parquet -> {parquet_path}")  
data_csv.write.mode("overwrite").parquet(parquet_path)
```

From the earlier .csv file that has been read by Spark, it is turned into a Parquet which is a columnar binary format. It is converted to allow for smaller size in comparison with csv, and better querying performance.

Preparing for Fact and Dimension table Conversion

```

INTEGER_DIMENSIONS = [
    "TP_DEPENDENCIA",
    "TP_LOCALIZACAO",
    "IN_AGUA_POTAVEL",
    "IN_ENERGIA_INEXISTENTE",
    "IN_ESGOTO_INEXISTENTE",
    "IN_BANHEIRO",
    "IN_BIBLIOTECA",
    "IN_REFEITORIO",
    "IN_COMPUTADOR",
    "IN_INTERNET",
    "IN_EQUIP_NENHUM",
]

DIMENSION_TABLES_CONFIG = {
    "DIM_LOCAL": {
        "fields": [
            {"field": "NO_UF", "type": "string"},
            {"field": "SG_UF", "type": "string"},
            {"field": "CO_UF", "type": "string"},
            {"field": "NO_MUNICIPIO", "type": "string"},
            {"field": "CO_MUNICIPIO", "type": "string"},
        ]
    }
}

DIMENSION_TABLES_CONFIG.update(
    {
        "DIM_" + dim.upper(): [{"field": dim, "type": "integer"}]
        for dim in INTEGER_DIMENSIONS
    }
)

```

Dimension table is declared based on the integer dimensions generated, becoming the single-column dimension via the name DIM <COLUMN>.

Missing Data, Outlier and Inconsistencies Handling

The INEP Censo Escolar microdata is a published, curated government dataset, not raw sensor or form data. INEP runs its own validation pipeline before releasing the ZIPs — range checks, referential consistency between the schools/classes/students files, encoding normalisation, and so on. The pipeline is built on the assumption that a national statistical agency has done that work before publishing. For a tutorial project, that assumption is reasonable.

The pipeline's role is structural, not corrective. The ETL is doing a specific job: reshape INEP's flat CSV into a star schema. It's not positioned as a data-cleaning layer — those are two different concerns, and mixing them would blur the responsibility of each stage. In a proper layered architecture you'd have a separate "raw → cleaned" stage before this "cleaned →

modelled" stage (the Bronze → Silver → Gold pattern in medallion architectures, or the Source → Staging → DWH split in classical DW design). The pipeline essentially

skips straight from raw to modelled because INEP's curation acts as the staging layer.

What happens structurally if the data were dirty. The pipeline isn't completely blind — it just doesn't act on problems. If a school row had a NULL CO_MUNICIPIO, that row would left-join to a NULL ID_DIM_LOCAL in the fact load, which Postgres would reject via the FK constraint, and the whole fact write would fail noisily. That's not graceful, but it's at least loud — you'd see it immediately in the Docker logs rather than silently producing a broken fact table. A production version would add a pre-load assertion (`data.filter(F.col("CO_MUNICIPIO").isNull()).count() == 0`) to catch that before it hits Postgres.

4.0 Modelling

4.1 System Architecture

4.2 Multidimensional Data Model

5.0 Evaluation

This section discusses the limitations and improvements in the pipeline.

5.1 Limitation

While The pipeline works well for this project, there are a few technical issues to keep in mind :

- Starts Over Every Time :

The system deletes and recreates the entire database every time it runs. There is no features to add the data for a new year, you need to reload all previous years from scratch

- Trusts The Source Data :

The pipeline assumed that the raw files were already perfect. For a good practice, we would suggest adding “checkpoints” to catch the errors such as missing values, etc before they reach the master database.

- Limited Capacity :

The data processing (Apache Spark) happens on a single computer. This may work for 2.79 million rows, but if the dataset grew to more than 50 million rows, we would need to use a multiple computer (a cluster) to handle the load

5.2 Improvement

This are the suggestion to make this project from just a tutorial to a production ready system

- Smart Updates :

Change the system so it can add a new year at a time without needing to reprocess the entire history. This will help to save a lot of time.

- Automated Quality Checks :

Use specialized tools to automatically detect any weird data or errors the moment they appear.

- Cloud Power :

Move the processing to a cloud service like Databrick. This will allow the system to handle data from every school in the world and no need to care about the storage.